

Chamadas de API do Codex no Coding_Machine

Pergunta: quantas chamadas de API o motor autônomo gastou por pacote do backlog, em que tentativa cada uma aconteceu e com que modelo. **Fonte:** `.autodev/state.db`, tabela `attempts` (o próprio motor grava uma linha por invocação). **Janela:** 27/09/2026 02:03 a 28/09/2026 17:42 — DEVFACTORY-001, 002 e 004 (a 003 foi planejada e nunca executada). **Data do relatório:** 28/09/2026. **Total no período: 92 chamadas do Codex**, 47 delas aprovadas (revisão + integração).

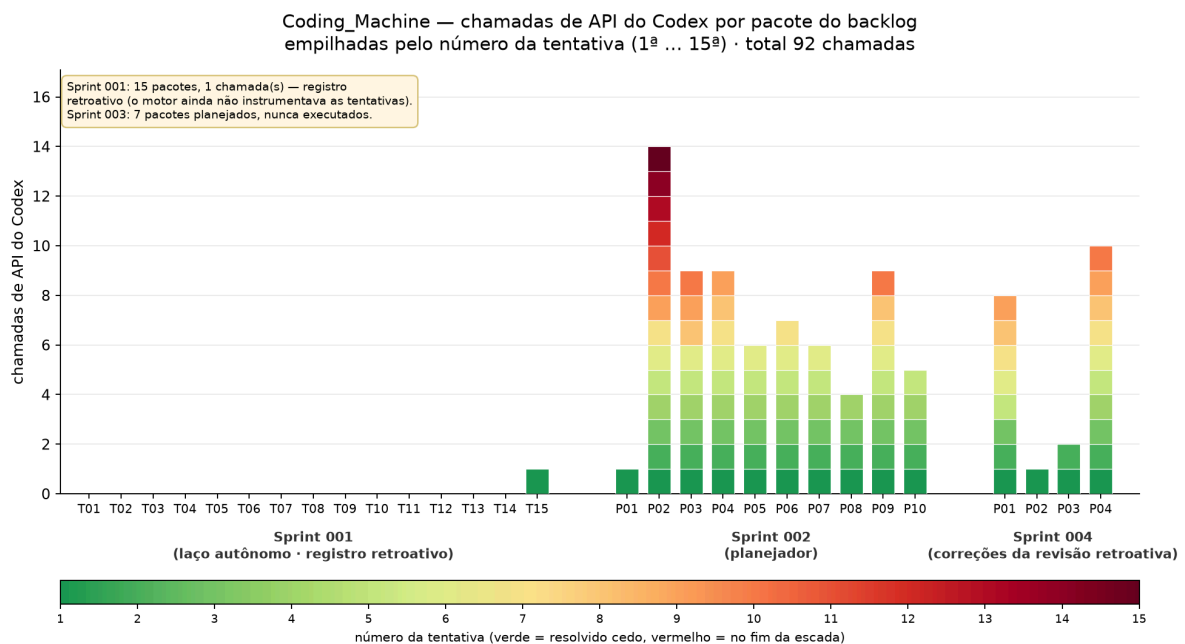
Avisos

1. **Chamada não é custo.** Cada linha conta **uma invocação** do agente; o motor não registra tokens, então este relatório mede chamadas, não gasto.
2. **A sprint 001 não é comparável.** Dos seus 15 pacotes, 14 são **registro retroativo** (agente `retroativo`, inserido em 27/09 02:03 para reconstruir o histórico) — **não são chamadas de API**. Só o T15 tem uma chamada real, e sem modelo registrado. É por isso que 14 colunas da sprint 001 estão vazias.
3. **A sprint 003 não aparece no gráfico:** 7 pacotes planejados, nenhum executado.
4. **A sprint 004 está em andamento** (4 de 4 pacotes já com chamadas; a P03 está aguardando cota do Codex) — os números dela ainda vão mudar.
5. **“Nª tentativa” não é o degrau da escada de modelos.** Falhas de infraestrutura (`CODEX_QUOTA`, `NETWORK_ERROR`, `ENVIRONMENT_ERROR`, `DEPENDENCY_ERROR`, `PERMISSION_REQUIRED`, `SECRET_REQUIRED`, `RED_ACTION_REQUIRED`) reprocessam **no mesmo modelo** por decisão de política — por isso `luna/low` reaparece em degraus altos. O degrau mede “quantas vezes tentou”, não “quão forte era o modelo”.
6. **A partir da 5ª tentativa o modelo é sempre o mesmo** (`astra/low`, tier 4): o mapa da escada satura em 5, então degraus 5 a 15 podem repetir o modelo do topo — e, nas classes do aviso 5, repetir o do fundo.
7. **3 combinação(ões) fora da escada declarada:** `(nenhum)/(nenhum)`, `gpt-5.6-luna/medium`, `gpt-5.6-terra/medium`. As chamadas `luna/medium` e `terra/medium` aconteceram em 27/09 entre 03:13 e 04:07, **antes** de a escada ser padronizada naquele mesmo dia — não são desvio de política.
8. **A numeração por pacote tem buracos.** Rearme por dependência integrada e reabertura por defeito de contrato removem/renomeiam tentativas, então 4 pacote(s) (P02 (sprint 002), P03 (sprint 002), P09 (sprint 002), P01 (sprint 004)) têm sequência descontínua — marcados com ▲ na tabela 1. Consequência para a leitura: o degrau 1 pode se repetir na história de um mesmo pacote, e “15 primeiras tentativas para 15 pacotes” é coincidência, não regra.

O que os números dizem

- **92 chamadas do Codex** em 15 pacotes com execução registrada. A mediana é de 6 chamadas por pacote e a média 6.1.
- **16.3% das chamadas são de 1ª tentativa** e 29.3% acontecem até a 2ª. Metade do gasto (4ª tentativa em diante) está na cauda: são poucos pacotes que consumiram a escada inteira.
- **3 pacote(s) resolveram com uma única chamada:** T15 (sprint 001), P01 (sprint 002), P02 (sprint 004).
- **Os campeões de gasto:** P02 (sprint 002, 14 chamadas), P04 (sprint 004, 10 chamadas), P03 (sprint 002, 9 chamadas). Os dois são casos conhecidos: a P02 da sprint 2 pedia reescrita de contrato de teste (o mesmo arquivo para tasks diferentes) e a P04 da sprint 4 gastou a escada consertando o próprio comando de teste — 10 tentativas, das quais o defeito era meu, não do agente (decisão D-23).
- **A cauda direita do gráfico 1 é o sintoma mais caro do período:** 5 chamadas em degraus 11 a 15, todas em pacotes que só destravaram quando o **defeito de motor** foi corrigido (D-19, D-24, D-25) — nenhuma delas é “o modelo errado tentando mais”.

Gráfico 1 — chamadas por pacote, empilhadas pela tentativa



Chamadas de API do Codex por pacote do backlog, empilhadas pelo número da tentativa

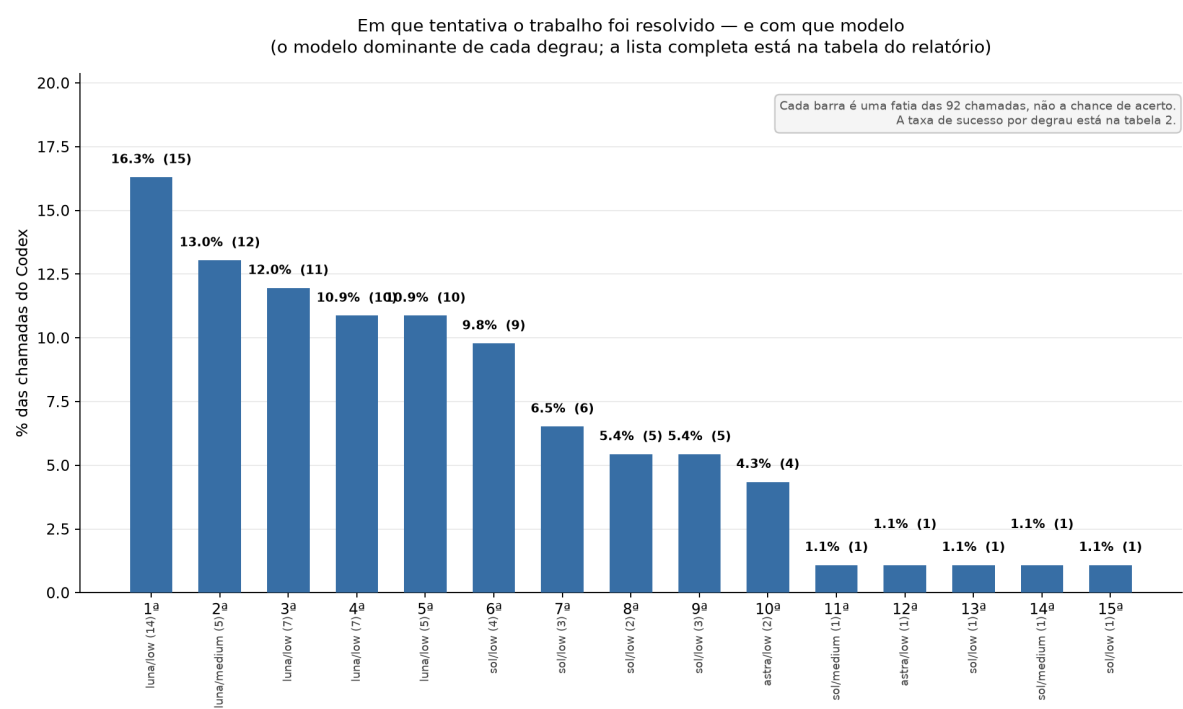
Cada coluna é um pacote; a altura é quantas vezes o Codex foi chamado nele; a cor diz **em que altura da escada** a chamada aconteceu (verde = cedo, vermelho = fim da escada). Total: 92 chamadas.

Tabela 1 — por pacote

sprint	pacote	chamadas	tentativas (1ª–última)	modelos usados
001	T15	1	1ª–1ª	(nenhum)/(nenhum)
002	P01	1	1ª–1ª	gpt-5.6-luna/low
002	P02	14	1ª–15ª ▲	gpt-5.6-luna/low, gpt-5.6-luna/medium, gpt-5.6-sol/low, gpt-5.6-sol/medium, gpt-6-astra/low
002	P03	9	1ª–10ª ▲	gpt-5.6-luna/low, gpt-5.6-sol/low, gpt-5.6-sol/medium
002	P04	9	1ª–9ª	gpt-5.6-luna/low, gpt-5.6-luna/medium, gpt-5.6-sol/low, gpt-5.6-sol/medium, gpt-6-astra/low
002	P05	6	1ª–6ª	gpt-5.6-luna/low, gpt-5.6-luna/medium, gpt-5.6-sol/low, gpt-5.6-terra/medium
002	P06	7	1ª–7ª	gpt-5.6-luna/low, gpt-5.6-sol/low, gpt-5.6-sol/medium, gpt-6-astra/low
002	P07	6	1ª–6ª	gpt-5.6-luna/low, gpt-5.6-luna/medium, gpt-5.6-sol/low, gpt-5.6-terra/medium
002	P08	4	1ª–4ª	gpt-5.6-luna/low, gpt-5.6-sol/low
002	P09	9	1ª–10ª ▲	gpt-5.6-luna/low, gpt-5.6-sol/low, gpt-5.6-sol/medium, gpt-6-astra/low

sprint	pacote	chamadas	tentativas (1ª-última)	modelos usados
002	P10	5	1ª-5ª	gpt-5.6-luna/low, gpt-5.6-luna/medium, gpt-5.6-sol/low
004	P01	8	1ª-9ª ▲	gpt-5.6-luna/low, gpt-5.6-sol/low, gpt-5.6-sol/medium, gpt-5.6-terra/low, gpt-6-astra/low
004	P02	1	1ª-1ª	gpt-5.6-luna/low
004	P03	2	1ª-2ª	gpt-5.6-luna/low, gpt-5.6-terra/low
004	P04	10	1ª-10ª	gpt-5.6-luna/low, gpt-5.6-sol/low, gpt-5.6-sol/medium, gpt-5.6-terra/low, gpt-6-astra/low

Gráfico 2 — distribuição por número de tentativa



Percentual das chamadas por número de tentativa, com o modelo dominante de cada degrau

Tabela 2 — por degrau

tentativa	chamadas	% das chamadas	% acumulado	aprovadas	modelos usados
1ª	15	16.3%	16.3%	7	gpt-5.6-luna/low (14), (nenhum)/(nenhum) (1)
2ª	12	13.0%	29.3%	7	gpt-5.6-luna/medium (5), gpt-5.6-luna/low (4), gpt-5.6-terra/low (3)
3ª	11	12.0%	41.3%	9	gpt-5.6-luna/low (7), gpt-5.6-terra/medium (2), gpt-5.6-sol/low (2)
4ª	10	10.9%	52.2%	8	gpt-5.6-luna/low (7), gpt-5.6-sol/low (2), gpt-5.6-sol/medium (1)
5ª	10	10.9%	63.0%	6	gpt-5.6-luna/low (5), gpt-5.6-sol/low (2), gpt-5.6-sol/medium (2), gpt-6-astra/low (1)
6ª	9	9.8%	72.8%	2	gpt-5.6-sol/low (4), gpt-5.6-luna/low (2), gpt-6-astra/low (2), gpt-5.6-sol/medium (1)
7ª	6	6.5%	79.3%	1	gpt-5.6-sol/low (3), gpt-6-astra/low (1), gpt-5.6-luna/low (1), gpt-5.6-terra/low (1)
8ª	5	5.4%	84.8%	1	gpt-5.6-sol/low (2), gpt-5.6-sol/medium (2), gpt-5.6-terra/low (1)
9ª	5	5.4%	90.2%	2	gpt-5.6-sol/low (3), gpt-6-astra/low (1),

tentativa	chamadas	% das chamadas	% acumulado	aprovadas	modelos usados
					gpt-5.6-sol/medium (1)
10 ^a	4	4.3%	94.6%	3	gpt-6-astra/low (2), gpt-5.6-sol/low (1), gpt-5.6-sol/medium (1)
11 ^a	1	1.1%	95.7%	0	gpt-5.6-sol/medium (1)
12 ^a	1	1.1%	96.7%	0	gpt-6-astra/low (1)
13 ^a	1	1.1%	97.8%	0	gpt-5.6-sol/low (1)
14 ^a	1	1.1%	98.9%	0	gpt-5.6-sol/medium (1)
15 ^a	1	1.1%	100.0%	1	gpt-5.6-sol/low (1)

A escada declarada × o que aconteceu

Escada de implementação no `models.yaml` (tentativa → modelo): 1^a `luna/low` · 2^a `terra/low` · 3^a `sol/low` · 4^a `sol/medium` · 5^a `astra/low`.

O que a base mostra é diferente em pontos importantes, e por motivos conhecidos:

1. **Reuso do mesmo modelo em falha de infraestrutura** (aviso 5) — a maior parte da diferença. Espera de cota e erro de ambiente não gastam escalonamento.
2. **Saturação depois da 5^a** (aviso 6): o mapa de escalonamento tem 5 entradas, então qualquer tentativa a partir da 5^a usa `astra/low`.
3. **Buracos e reinícios na numeração** (aviso 8) fazem o mesmo degrau aparecer com modelos diferentes conforme o momento do pacote — não é troca de política.
4. **5 chamadas anteriores à padronização** da própria escada (aviso 7).

Arquivos gerados e proveniência

arquivo	o que é
REPORT-TENTATIVAS-CODEX.md	este relatório
report/histograma-tentativas.png	gráfico 1
report/distribuicao-tentativas.png	gráfico 2
report/dados.json	agregação crua usada no texto e nos gráficos
report/gerar_relatorio_tentativas.py	gera os três a partir de <code>.autodev/state.db</code>
report/relatorio-tentativas.html / .pdf	HTML autocontido e PDF A4

Reproduzir:

```
cd ~/Code/Coding_Machine
~/hermes/cache/scratch/venv_report/bin/python report/gerar_relatorio_tentativas.py
pandoc REPORT-TENTATIVAS-CODEX.md -o report/relatorio-tentativas.html --standalone \
  --embed-resources --resource-path=.:report --css=report/estilo-relatorio.css \
  --metadata lang=pt-BR
chromium --headless=new --disable-gpu --no-sandbox --user-data-dir=/tmp/chrome-pdf \
  --virtual-time-budget=30000 --no-pdf-header-footer \
  --print-to-pdf="$PWD/report/relatorio-tentativas.pdf" \
  "file://$PWD/report/relatorio-tentativas.html"
```

O venv `~/hermes/cache/scratch/venv_report` existe só para o `matplotlib` (o venv do projeto não tem gráficos) e é descartável: `uv venv + uv pip install matplotlib`.